

Digital Image Processing Assignment 2

```
%% Digital Image Processing Assignment 2
% **Name:** Naga Prem Sai Nellure
% **Email:** nnellure2022@fau.edu
% **Professor:** Dr. Velibor Adzic

%% Question 1: Binary Mask and Filtering
% *Description:*
% This section creates a binary mask for the region of interest and applies
% Gaussian,
% Average, Laplacian, and Prewitt filters to the image.
%
% *Code:*
img = imread('Bat.jpg');
gray_img = rgb2gray(img);
figure, imshow(gray_img);
title('Original Grayscale Image');
```

Original Grayscale Image



```
threshold_value = 100;
binary_mask = imbinarize(gray_img, threshold_value/255);
figure, imshow(binary_mask);
title('Binary Mask');
```

Binary Mask



```
h_gaussian = fspecial('gaussian', [5,5], 2);  
img_gaussian = imfilter(gray_img, h_gaussian);  
figure, imshow(img_gaussian);  
title('Gaussian Filtered Image');
```

Gaussian Filtered Image



```
disp('Histogram for Gaussian Filtered Image:');
```

Histogram for Gaussian Filtered Image:

```
disp(histcounts(img_gaussian));
```

10853	1011795	51296	26301	17938	15555	14948	13242
-------	---------	-------	-------	-------	-------	-------	-------

110

```
h_average = fspecial('average', [5,5]);  
img_avg = imfilter(gray_img, h_average);  
figure, imshow(img_avg);  
title('Average Filtered Image');
```

Average Filtered Image



```
disp('Histogram for Average Filtered Image:');
```

Histogram for Average Filtered Image:

```
disp(histcounts(img_avg));
```

9519	1008839	50084	26133	17822	15484	14868	12960
------	---------	-------	-------	-------	-------	-------	-------

108

```
h_laplacian = fspecial('laplacian', 0.2);  
img_laplacian = imfilter(gray_img, h_laplacian);  
figure, imshow(img_laplacian, []);  
title('Laplacian Filtered Image');
```

Laplacian Filtered Image



```
disp('Histogram for Laplacian Filtered Image:');
```

Histogram for Laplacian Filtered Image:

```
disp(histcounts(img_laplacian));
```

4032366	0	0	0	0	456887	0	0
---------	---	---	---	---	--------	---	---

```
img_prewitt = edge(gray_img, 'prewitt');  
figure, imshow(img_prewitt);  
title('Prewitt Filtered Image');
```

Prewitt Filtered Image



```
disp('Histogram for Prewitt Filtered Image:');
```

Histogram for Prewitt Filtered Image:

```
disp(histcounts(img_prewitt));
```

5766828 234944

```
%Explanation
```

```
disp('Explanation and Observations:');
```

Explanation and Observations:

```
disp('1. The original image is converted to grayscale to simplify the  
analysis.');
```

1. The original image is converted to grayscale to simplify the analysis.

```
disp('2. A binary mask is created using a threshold to isolate regions of  
interest.');
```

2. A binary mask is created using a threshold to isolate regions of interest.

```
disp('3. Gaussian and Average filters are applied to reduce noise and smooth  
the image.');
```

3. Gaussian and Average filters are applied to reduce noise and smooth the image.

```
disp('4. Laplacian and Prewitt filters are used to enhance and detect edges, respectively.');
```

4. Laplacian and Prewitt filters are used to enhance and detect edges, respectively.

```
disp('5. The effects of these filters are visualized and can be analyzed by comparing the output images and histograms.');
```

5. The effects of these filters are visualized and can be analyzed by comparing the output images and histograms.

```
disp('6. Histograms provide a quantitative view of the pixel intensity distribution after each filtering step, offering insights into the filter effects.');
```

6. Histograms provide a quantitative view of the pixel intensity distribution after each filtering step, offering insights into the filter effects.

```
%% Question 2: Floyd-Steinberg and Jarvis-Judice-Ninke Dithering
% *Description:*
% Floyd-Steinberg and Jarvis-Judice-Ninke dithering are applied to reduce
% the color depth of the image while maintaining visual quality.
%
% *Code:*
img = imread('Bat.jpg');
gray_img = rgb2gray(img);
gray_img = im2double(gray_img);

floyd_steinberg = [0 0 7; 3 5 1] / 16;
jarvis_judice_ninke = [0 0 0 7 5; 3 5 7 5 3; 1 3 5 3 1] / 48;

floyd_img = ditheringAlgorithm(gray_img, floyd_steinberg);
jarvis_img = ditheringAlgorithm(gray_img, jarvis_judice_ninke);

figure, imshow(gray_img), title('Original Grayscale Image');
```

Original Grayscale Image



```
figure, imshow(floyd_img), title('Floyd-Steinberg Dithered Image');
```


Floyd-Steinberg Dithered Image

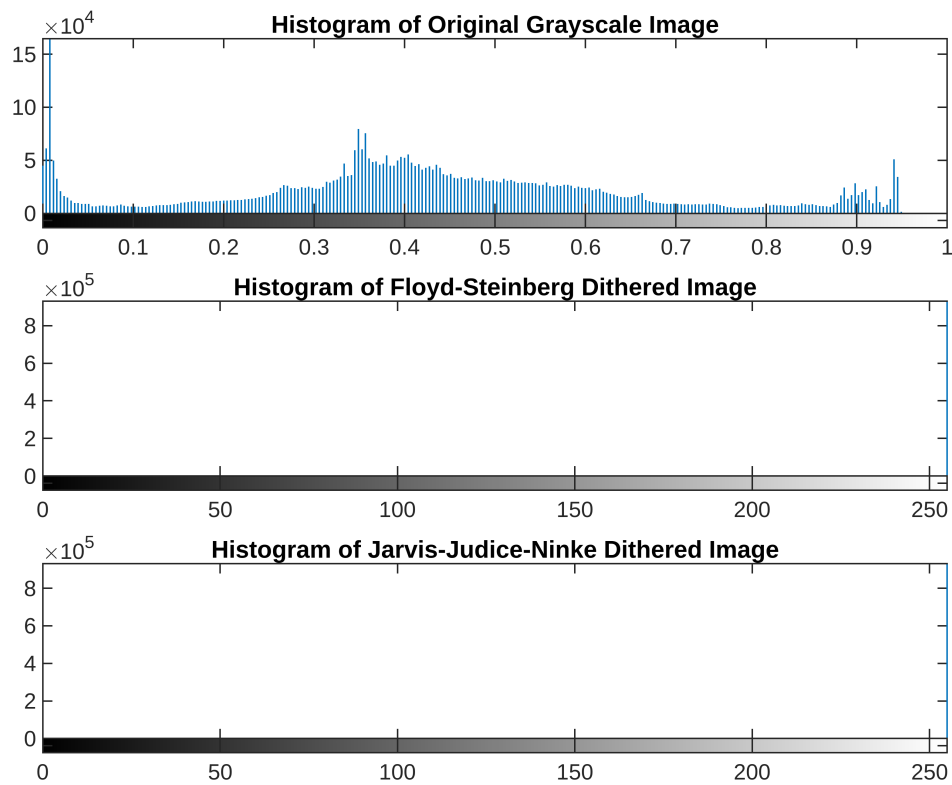


```
figure, imshow(jarvis_img), title('Jarvis-Judice-Ninke Dithered Image');
```

Jarvis-Judice-Ninke Dithered Image



```
figure;  
subplot(3,1,1), imhist(gray_img), title('Histogram of Original Grayscale  
Image');  
subplot(3,1,2), imhist(floyd_img), title('Histogram of Floyd-Steinberg  
Dithered Image');  
subplot(3,1,3), imhist(jarvis_img), title('Histogram of Jarvis-Judice-Ninke  
Dithered Image');
```



```
% Function to apply dithering based on input image and dither matrix
function output = ditheringAlgorithm(img, matrix)
    [rows, cols] = size(img);
    output = zeros(rows, cols);
    for i = 1:rows
        for j = 1:cols
            old_pixel = img(i, j);
            new_pixel = round(255 * old_pixel);
            output(i, j) = new_pixel;
            error = old_pixel - new_pixel/255;
            [mat_rows, mat_cols] = size(matrix);
            for x = 1:mat_rows
                for y = 1:mat_cols
                    ix = i + x - 2;
                    jy = j + y - 2 - floor(mat_cols/2);
                    if ix > 0 && ix <= rows && jy > 0 && jy <= cols
                        output(ix, jy) = output(ix, jy) + error * matrix(x,
y);
                    end
                end
            end
        end
    end
    output = uint8(output * 255);
```

```
end
```

```
% Explanation
```

```
disp('Explanation and Observations:');
```

Explanation and Observations:

```
disp('The Floyd-Steinberg dithering uses a 4-neighbor diffusion matrix and is primarily used for monochrome printers. The error is propagated to the neighboring pixels that are yet to be processed. This method is sensitive to the ordering of pixels.');
```

The Floyd-Steinberg dithering uses a 4-neighbor diffusion matrix and is primarily used for monochrome printers.

```
disp('The Jarvis-Judice-Ninke dithering, on the other hand, uses a larger diffusion matrix with more neighbors, thus it tends to provide a smoother gradient in the dithered output at the cost of higher computational expense. It is effective for images with gradual tonal changes.');
```

The Jarvis-Judice-Ninke dithering, on the other hand, uses a larger diffusion matrix with more neighbors, thus it tends to provide a smoother gradient in the dithered output at the cost of higher computational expense.

```
disp('Both dithering techniques alter the image by distributing quantization errors across neighboring pixels, thereby reducing the appearance of quantization errors and improving the visual quality of the dithered images. The histograms demonstrate the redistribution of pixel intensities due to each method.');
```

Both dithering techniques alter the image by distributing quantization errors across neighboring pixels, thereby reducing the appearance of quantization errors and improving the visual quality of the dithered images.

```
disp('Numerical histogram values for both dithering techniques would further quantify the changes in pixel distribution and help in assessing the effectiveness of each dithering approach in different scenarios.');
```

Numerical histogram values for both dithering techniques would further quantify the changes in pixel distribution and help in assessing the effectiveness of each dithering approach in different scenarios.

```
%% Question 3: Kuwahara Filter
```

```
% *Description:*
```

```
% The Kuwahara filter is applied with a 5x5 window to reduce noise while preserving edges.
```

```
%
```

```
% *Code:*
```

```
function output = kuwaharaFilter(img, ksize)
    if mod(ksize, 2) == 0
        error('Kernel size must be odd.');
```

```
    padSize = floor(ksize/2);
    paddedImg = padarray(img, [padSize padSize], 'replicate', 'both');
```

```
    [rows, cols] = size(img);
    output = zeros(rows, cols);
```

```

for i = 1:rows
    for j = 1:cols

        center_i = i + padSize;
        center_j = j + padSize;

        sub1 = paddedImg(center_i-padSize:center_i-1, center_j-
padSize:center_j-1);
        sub2 = paddedImg(center_i-padSize:center_i-1,
center_j+1:center_j+padSize);
        sub3 = paddedImg(center_i+1:center_i+padSize, center_j-
padSize:center_j-1);
        sub4 = paddedImg(center_i+1:center_i+padSize,
center_j+1:center_j+padSize);

        means = [mean(sub1(:)), mean(sub2(:)), mean(sub3(:)),
mean(sub4(:))];
        vars = [var(double(sub1(:))), var(double(sub2(:))),
var(double(sub3(:))), var(double(sub4(:)))];

        [~, idx] = min(vars);

        output(i, j) = means(idx);
    end
end

output = uint8(255 * mat2gray(output));
end

img = imread('Bat.jpg');
gray_img = rgb2gray(img);
filtered_img = kuwaharaFilter(gray_img, 5);
figure;
subplot(1,2,1), imshow(gray_img), title('Original Grayscale Image');
subplot(1,2,2), imshow(filtered_img), title('Kuwahara Filtered Image');

```

Original Grayscale Image



Kuwahara Filtered Image



```
% Load the image one more time to check with the difference with kuwahara
img = imread('Bat.jpg');
gray_img = rgb2gray(img);
filtered_img = kuwaharaFilter(gray_img, 5);
figure;
subplot(1,3,1), imshow(gray_img), title('Original Grayscale Image');
subplot(1,3,2), imshow(filtered_img), title('Kuwahara Filtered Image');
diff_img = abs(double(gray_img) - double(filtered_img));
subplot(1,3,3), imshow(diff_img, []), title('Difference Image');
```

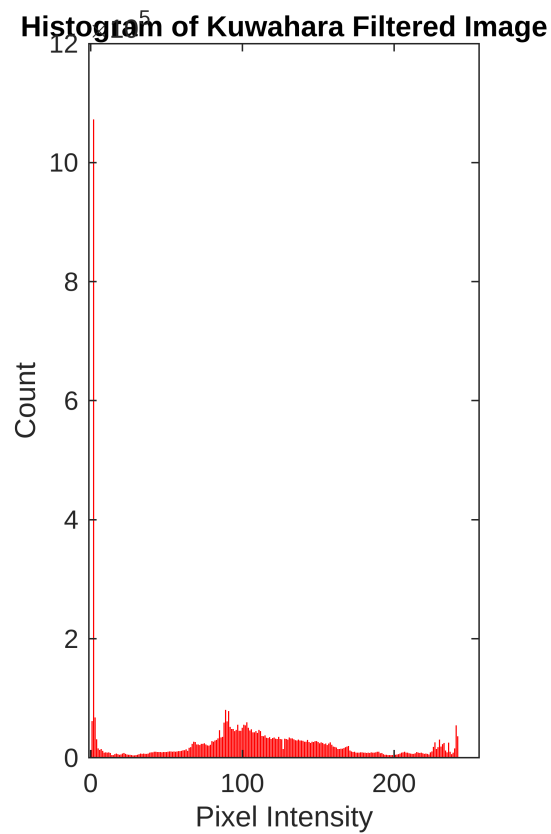
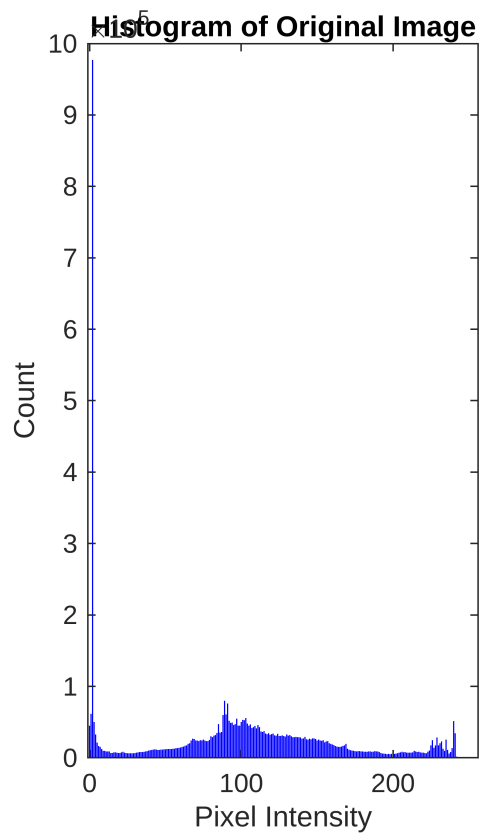
Original Grayscale Image Kuwahara Filtered Image



Difference Image



```
[hist_orig, orig_bins] = imhist(gray_img);  
[hist_filt, filt_bins] = imhist(filtered_img);  
  
figure;  
subplot(1,2,1);  
bar(orig_bins, hist_orig, 'b');  
title('Histogram of Original Image');  
xlabel('Pixel Intensity'); ylabel('Count');  
  
subplot(1,2,2);  
bar(filt_bins, hist_filt, 'r');  
title('Histogram of Kuwahara Filtered Image');  
xlabel('Pixel Intensity'); ylabel('Count');
```



```
disp('Histogram Values for Original Image:');
```

Histogram Values for Original Image:

```
disp([orig_bins, hist_orig]);
```

0	45146
1	61503
2	976939
3	50033
4	32732
5	21179
6	16502
7	15117
8	12287
9	9819
10	9822
11	8988
12	9001
13	9019
14	6758
15	6805
16	7482
17	7603
18	7402
19	6870
20	6884
21	7630
22	8525
23	7241
24	6858

25	6505
26	6552
27	6401
28	6320
29	6348
30	6711
31	7080
32	7623
33	7870
34	7970
35	8058
36	8298
37	8764
38	9203
39	10152
40	10565
41	10844
42	11281
43	11665
44	11384
45	11087
46	11223
47	11489
48	11514
49	11861
50	11886
51	12094
52	12152
53	12372
54	12474
55	12686
56	12816
57	13411
58	13663
59	14051
60	14590
61	15407
62	15598
63	16898
64	17497
65	19316
66	20278
67	24182
68	26728
69	26372
70	24021
71	24076
72	23150
73	24827
74	24321
75	25308
76	24122
77	23427
78	23411
79	25135
80	29946
81	29067
82	31201
83	31816
84	34876
85	46967
86	35317
87	36214
88	59680

89	79699
90	60626
91	75837
92	51946
93	48654
94	49158
95	45979
96	47167
97	54715
98	45193
99	44995
100	50013
101	53282
102	52512
103	55564
104	48016
105	44850
106	46541
107	41489
108	43041
109	44533
110	41353
111	45978
112	43077
113	37016
114	36056
115	37460
116	33757
117	33002
118	34288
119	32452
120	33103
121	33839
122	31487
123	31047
124	33742
125	30544
126	30628
127	31493
128	30365
129	29515
130	32772
131	30822
132	31646
133	30263
134	28678
135	28980
136	29439
137	28693
138	28750
139	28623
140	26662
141	26970
142	29471
143	25877
144	25406
145	26740
146	26048
147	27164
148	27016
149	26119
150	23989
151	25523
152	24340

153	23833
154	24497
155	21858
156	22915
157	23459
158	20437
159	19636
160	18520
161	17848
162	16435
163	15797
164	15435
165	15305
166	15609
167	16424
168	17545
169	19503
170	12880
171	11540
172	10788
173	10161
174	9811
175	9296
176	9088
177	9202
178	9270
179	8991
180	8816
181	8639
182	8677
183	8566
184	8792
185	8794
186	8526
187	8430
188	9319
189	8953
190	8782
191	7943
192	6967
193	6032
194	5886
195	5415
196	5203
197	5292
198	5270
199	5365
200	5476
201	5617
202	6128
203	6364
204	7165
205	7663
206	8368
207	7708
208	7908
209	7281
210	7195
211	7055
212	7068
213	7646
214	9601
215	8701
216	8143

217	8683
218	7837
219	7162
220	7034
221	6857
222	6269
223	8541
224	10028
225	17196
226	24605
227	13802
228	17355
229	28629
230	17394
231	20361
232	22816
233	12701
234	9623
235	25637
236	10867
237	6125
238	8124
239	13672
240	51180
241	34488
242	1593
243	880
244	478
245	254
246	152
247	104
248	66
249	45
250	31
251	17
252	24
253	19
254	16
255	1

```
disp('Histogram Values for Kuwahara Filtered Image:');
```

Histogram Values for Kuwahara Filtered Image:

```
disp([filt_bins, hist_filt]);
```

0	9691
1	61515
2	1072473
3	67776
4	30883
5	16018
6	13058
7	14793
8	11876
9	8649
10	9001
11	8492
12	9112
13	8279
14	4717
15	4760
16	5903

17	6871
18	6340
19	5181
20	5492
21	6948
22	8369
23	6386
24	5352
25	4940
26	4843
27	4569
28	3955
29	4301
30	4600
31	5085
32	6321
33	6939
34	6817
35	6895
36	6484
37	6523
38	7308
39	8508
40	8996
41	9048
42	9976
43	10210
44	9815
45	9527
46	9681
47	9243
48	9518
49	9676
50	9890
51	10397
52	10509
53	10433
54	10369
55	10755
56	10137
57	10592
58	11420
59	11344
60	11708
61	12885
62	12532
63	14111
64	11678
65	16786
66	17911
67	23245
68	26870
69	26384
70	22546
71	22277
72	21249
73	23345
74	23676
75	24545
76	22500
77	20691
78	20575
79	21836
80	27826

81	26928
82	28932
83	30000
84	32636
85	46127
86	33904
87	34905
88	59300
89	80406
90	61397
91	78624
92	52105
93	48299
94	48230
95	45085
96	46827
97	55395
98	45517
99	45480
100	50655
101	55773
102	54416
103	59578
104	50217
105	45658
106	48006
107	42559
108	42951
109	44612
110	41813
111	47075
112	45004
113	36479
114	36127
115	37549
116	33814
117	33055
118	34698
119	31331
120	32957
121	34279
122	32323
123	31826
124	35210
125	31615
126	31294
127	15011
128	31951
129	31703
130	30494
131	34286
132	32377
133	32380
134	31125
135	29640
136	29181
137	30802
138	29217
139	28984
140	28494
141	27210
142	26802
143	30256
144	26495

145	25168
146	27066
147	26681
148	28108
149	28254
150	26641
151	24380
152	25884
153	25016
154	22749
155	23737
156	21219
157	24177
158	26153
159	21848
160	18748
161	18448
162	17506
163	14949
164	14595
165	15208
166	15346
167	16060
168	17624
169	19012
170	20066
171	12129
172	10468
173	9480
174	10045
175	8833
176	8756
177	8602
178	9002
179	9278
180	8803
181	8686
182	8249
183	8750
184	8091
185	9192
186	8689
187	8432
188	9214
189	10405
190	10337
191	7813
192	8238
193	5950
194	5093
195	5161
196	4729
197	4553
198	4735
199	4817
200	4724
201	5252
202	5404
203	5938
204	6878
205	8488
206	9187
207	10065
208	8686

209	8857
210	7497
211	7320
212	6684
213	6616
214	7616
215	9471
216	8793
217	7971
218	8418
219	7485
220	6724
221	6916
222	6654
223	5824
224	8947
225	10738
226	18300
227	26100
228	14616
229	17698
230	30414
231	18962
232	23085
233	24726
234	12285
235	9178
236	25361
237	10172
238	5869
239	8754
240	16014
241	54591
242	36001
243	716
244	175
245	128
246	62
247	29
248	13
249	14
250	15
251	7
252	15
253	10
254	6
255	16

% Explanation

```
disp('Explanation of Kuwahara Filter Process and Observations:');
```

Explanation of Kuwahara Filter Process and Observations:

```
disp('The Kuwahara filter, applied here with a 5x5 window, is designed to  
preserve edge features while reducing noise in homogeneous areas.');
```

The Kuwahara filter, applied here with a 5x5 window, is designed to preserve edge features while reducing

```
disp('This filter works by dividing the window into four overlapping sub-  
windows. Each sub-windows variance and mean are calculated.');
```


This filter works by dividing the window into four overlapping sub-windows. Each sub-windows variance and

```
disp('The output pixel is set to the mean of the sub-window with the lowest  
variance, ensuring that edge details are preserved while smoothing areas  
without strong edges.');
```

The output pixel is set to the mean of the sub-window with the lowest variance, ensuring that edge details

```
disp('This method is particularly effective in images with clear and smooth  
areas separated by sharp edges, making it ideal for images with significant  
texture variation.');
```

This method is particularly effective in images with clear and smooth areas separated by sharp edges, making

```
disp('The difference image highlights where the filter has made significant  
changes, typically around the edges and in regions with noise.');
```

The difference image highlights where the filter has made significant changes, typically around the edges

```
disp('Histogram comparisons provide a quantitative measure of the changes,  
showing how pixel intensities are distributed before and after filtering.');
```

Histogram comparisons provide a quantitative measure of the changes, showing how pixel intensities are distributed

```
disp('Such visual and numerical analysis helps in understanding the filter's  
impact on image aesthetics and quality.');
```

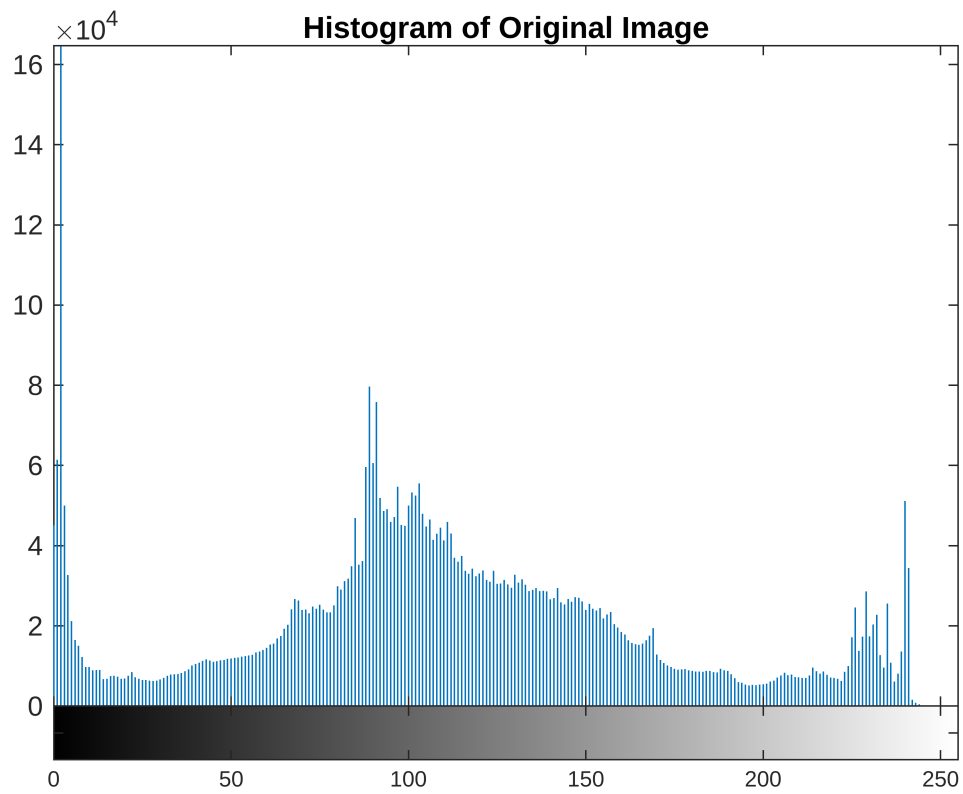
Such visual and numerical analysis helps in understanding the filter's impact on image aesthetics and quality

```
%% Question 4: Fourier Transform, Butterworth, and Gaussian Filters  
% *Description:*  
% Fourier Transform is applied to analyze the frequency domain of the image  
and two filters are implemented.  
%  
% *Code:*  
img = imread('Bat.jpg');  
gray_img = rgb2gray(img);  
figure, imshow(gray_img);  
title('Original Grayscale Image');
```

Original Grayscale Image

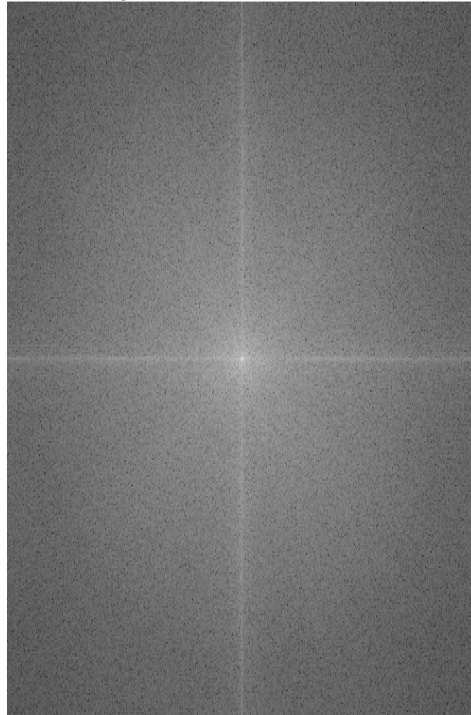


```
figure;  
imhist(gray_img);  
title('Histogram of Original Image');
```



```
F = fft2(double(gray_img));  
F_shifted = fftshift(F);  
magnitude_spectrum = log(1+abs(F_shifted));  
figure, imshow(magnitude_spectrum, []);  
title('Magnitude Spectrum of Fourier Transform');
```

Magnitude Spectrum of Fourier Transform

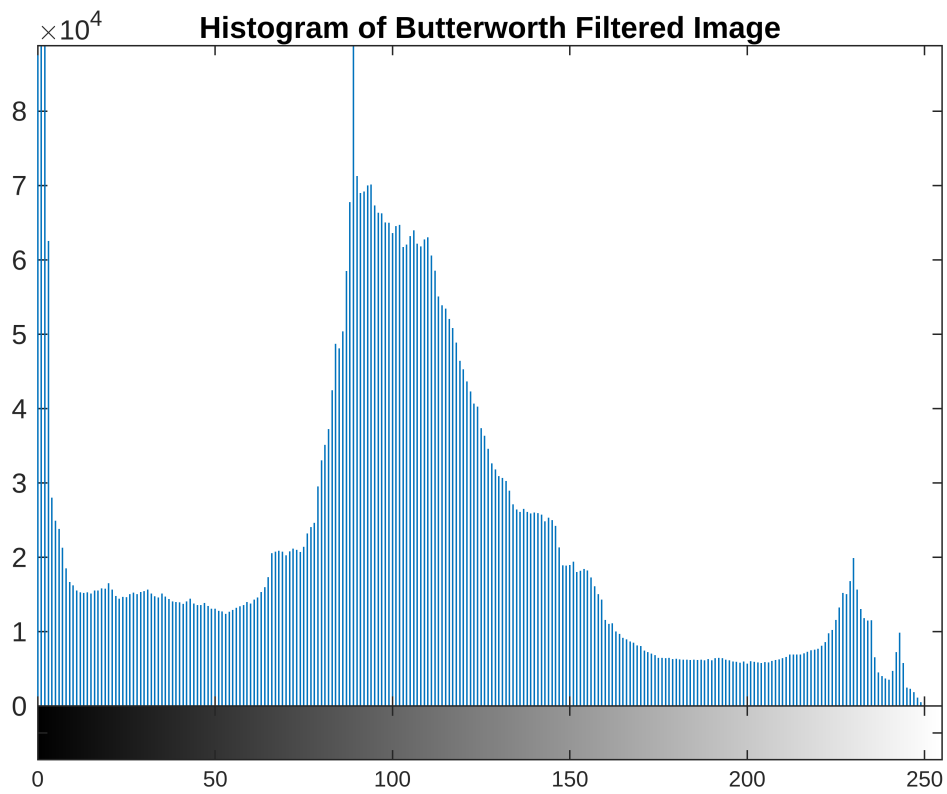


```
[u, v] = size(gray_img);  
[U, V] = meshgrid(-v/2:v/2-1, -u/2:u/2-1);  
D = sqrt(U.^2 + V.^2);  
D0 = 25;  
n = 2;  
H = 1 ./ (1 + (D / D0).^(2*n));  
G = H .* F_shifted;  
G_shifted = ifftshift(G);  
filtered_img_butter = real(ifft2(G_shifted));  
figure, imshow(filtered_img_butter, []);  
title('Butterworth Filtered Image');
```

Butterworth Filtered Image



```
figure, imhist(uint8(filtered_img_butter));  
title('Histogram of Butterworth Filtered Image');
```

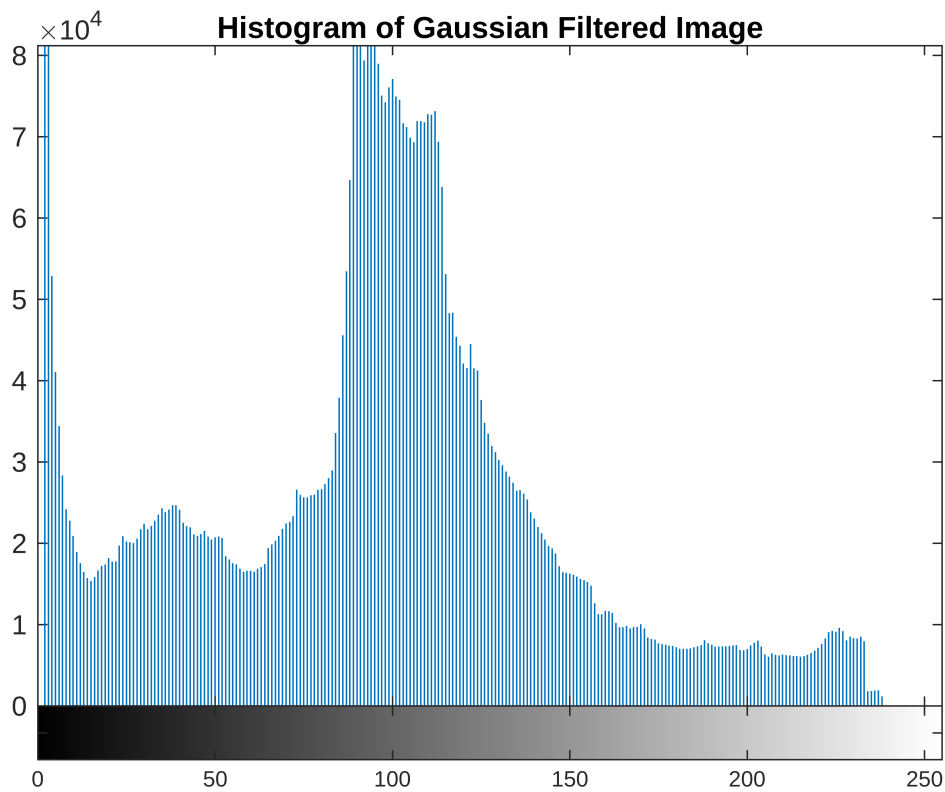


```
sigma = 10;  
H_gaussian = exp(-(D.^2) / (2*sigma^2));  
G_gaussian = H_gaussian .* F_shifted;  
G_gaussian_shifted = ifftshift(G_gaussian);  
filtered_img_gaussian = real(ifft2(G_gaussian_shifted));  
figure, imshow(filtered_img_gaussian, []);  
title('Gaussian Filtered Image');
```

Gaussian Filtered Image



```
figure, imhist(uint8(filtered_img_gaussian));  
title('Histogram of Gaussian Filtered Image');
```



```
mse_butter = immse(double(gray_img), double(filtered_img_butter));
psnr_butter = psnr(double(filtered_img_butter), double(gray_img));

mse_gaussian = immse(double(gray_img), double(filtered_img_gaussian));
psnr_gaussian = psnr(double(filtered_img_gaussian), double(gray_img));
```

```
fprintf('MSE for Butterworth Filtered Image: %f\n', mse_butter);
```

```
MSE for Butterworth Filtered Image: 821.065600
```

```
fprintf('PSNR for Butterworth Filtered Image: %f dB\n', psnr_butter);
```

```
PSNR for Butterworth Filtered Image: -29.143779 dB
```

```
fprintf('MSE for Gaussian Filtered Image: %f\n', mse_gaussian);
```

```
MSE for Gaussian Filtered Image: 1231.544830
```

```
fprintf('PSNR for Gaussian Filtered Image: %f dB\n', psnr_gaussian);
```

```
PSNR for Gaussian Filtered Image: -30.904502 dB
```

```
% Explanation
```

```
disp('Explanation of Fourier Transform, Butterworth, and Gaussian Filters:');
```


Explanation of Fourier Transform, Butterworth, and Gaussian Filters:

```
disp('Fourier Transform is used to analyze the frequency components of an image. The magnitude spectrum displays the intensity of the frequencies. Areas with high variations, such as edges, correspond to higher values in the spectrum.');
```

Fourier Transform is used to analyze the frequency components of an image. The magnitude spectrum displays

```
disp('Butterworth filter, a type of low-pass filter, is characterized by its smoothness and lack of ripples in the frequency magnitude response. This smoothness is due to the order of the filter and the cut-off frequency, which controls the transition from passband to stopband.');
```

Butterworth filter, a type of low-pass filter, is characterized by its smoothness and lack of ripples in t

```
disp('Gaussian filter, also a low-pass filter, uses a Gaussian function for its impulse response. It is highly effective in reducing high-frequency components, thus smoothing the image. The standard deviation of the Gaussian function controls the extent of smoothing.');
```

Gaussian filter, also a low-pass filter, uses a Gaussian function for its impulse response. It is highly e

```
disp('The filters reduce noise and details, with the Butterworth filter providing a smoother transition compared to the sharper transition of the Gaussian filter.');
```

The filters reduce noise and details, with the Butterworth filter providing a smoother transition compared

```
disp('Comparing the results, the Butterworth filter maintains more of the low-frequency components, while the Gaussian filter attenuates higher frequencies more aggressively.');
```

Comparing the results, the Butterworth filter maintains more of the low-frequency components, while the Ga

```
disp('Histograms and the MSE (Mean Squared Error) and PSNR (Peak Signal-to-Noise Ratio) metrics help quantify the effects of these filters on the image quality.');
```

Histograms and the MSE (Mean Squared Error) and PSNR (Peak Signal-to-Noise Ratio) metrics help quantify th

```
%% Question 5: Quantization to 32 Levels
% *Description:*
% This section performs quantization on the grayscale image to reduce it to 32 levels using `imresize`.
%
% *Code:*
img = imread('Bat.jpg');
gray_img = rgb2gray(img);
figure, imshow(gray_img);
title('Original Grayscale Image');
```

Original Grayscale Image

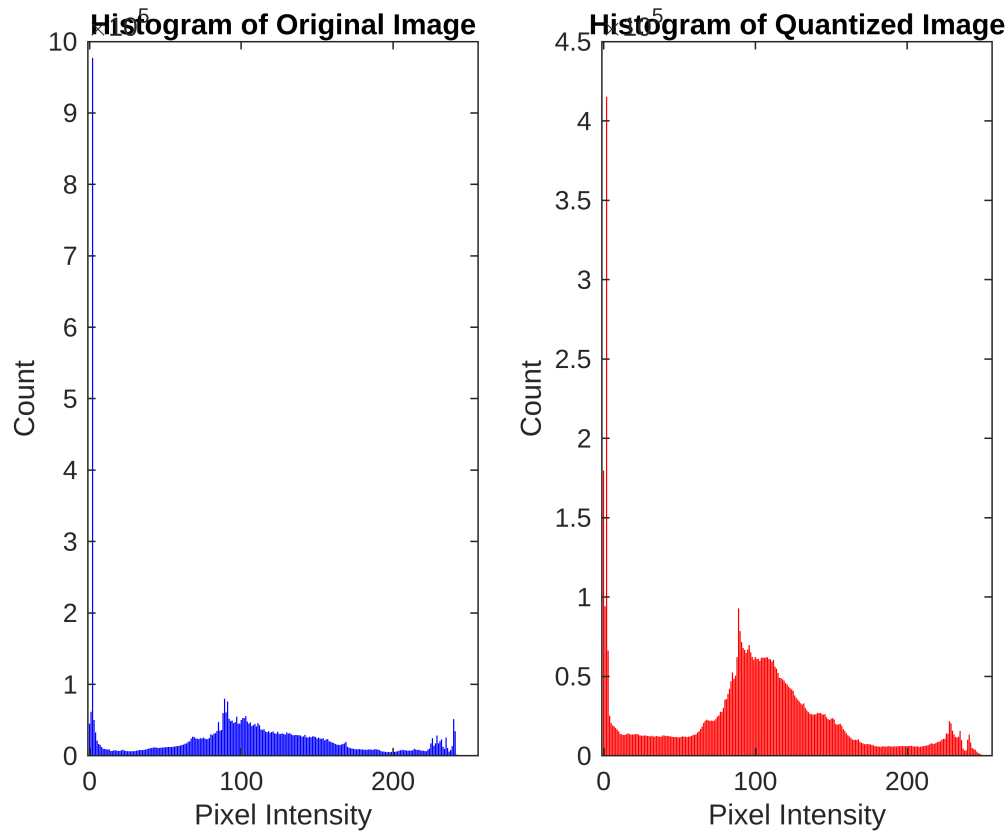


```
quantized_img = imresize(imresize(gray_img, 1/32), 32);  
figure, imshow(quantized_img);  
title('Quantized to 32 Levels Image');
```

Quantized to 32 Levels Image



```
[hist_orig, orig_bins] = imhist(gray_img);  
[hist_quant, quant_bins] = imhist(quantized_img);  
  
figure;  
subplot(1,2,1);  
bar(orig_bins, hist_orig, 'b');  
title('Histogram of Original Image');  
xlabel('Pixel Intensity'); ylabel('Count');  
  
subplot(1,2,2);  
bar(quant_bins, hist_quant, 'r');  
title('Histogram of Quantized Image');  
xlabel('Pixel Intensity'); ylabel('Count');
```



```
disp('Histogram Values for Original Image:');
```

Histogram Values for Original Image:

```
disp([orig_bins, hist_orig]);
```

0	45146
1	61503
2	976939
3	50033
4	32732
5	21179
6	16502
7	15117
8	12287
9	9819
10	9822
11	8988
12	9001
13	9019
14	6758
15	6805
16	7482
17	7603
18	7402
19	6870
20	6884
21	7630
22	8525
23	7241
24	6858

25	6505
26	6552
27	6401
28	6320
29	6348
30	6711
31	7080
32	7623
33	7870
34	7970
35	8058
36	8298
37	8764
38	9203
39	10152
40	10565
41	10844
42	11281
43	11665
44	11384
45	11087
46	11223
47	11489
48	11514
49	11861
50	11886
51	12094
52	12152
53	12372
54	12474
55	12686
56	12816
57	13411
58	13663
59	14051
60	14590
61	15407
62	15598
63	16898
64	17497
65	19316
66	20278
67	24182
68	26728
69	26372
70	24021
71	24076
72	23150
73	24827
74	24321
75	25308
76	24122
77	23427
78	23411
79	25135
80	29946
81	29067
82	31201
83	31816
84	34876
85	46967
86	35317
87	36214
88	59680

89	79699
90	60626
91	75837
92	51946
93	48654
94	49158
95	45979
96	47167
97	54715
98	45193
99	44995
100	50013
101	53282
102	52512
103	55564
104	48016
105	44850
106	46541
107	41489
108	43041
109	44533
110	41353
111	45978
112	43077
113	37016
114	36056
115	37460
116	33757
117	33002
118	34288
119	32452
120	33103
121	33839
122	31487
123	31047
124	33742
125	30544
126	30628
127	31493
128	30365
129	29515
130	32772
131	30822
132	31646
133	30263
134	28678
135	28980
136	29439
137	28693
138	28750
139	28623
140	26662
141	26970
142	29471
143	25877
144	25406
145	26740
146	26048
147	27164
148	27016
149	26119
150	23989
151	25523
152	24340

153	23833
154	24497
155	21858
156	22915
157	23459
158	20437
159	19636
160	18520
161	17848
162	16435
163	15797
164	15435
165	15305
166	15609
167	16424
168	17545
169	19503
170	12880
171	11540
172	10788
173	10161
174	9811
175	9296
176	9088
177	9202
178	9270
179	8991
180	8816
181	8639
182	8677
183	8566
184	8792
185	8794
186	8526
187	8430
188	9319
189	8953
190	8782
191	7943
192	6967
193	6032
194	5886
195	5415
196	5203
197	5292
198	5270
199	5365
200	5476
201	5617
202	6128
203	6364
204	7165
205	7663
206	8368
207	7708
208	7908
209	7281
210	7195
211	7055
212	7068
213	7646
214	9601
215	8701
216	8143

217	8683
218	7837
219	7162
220	7034
221	6857
222	6269
223	8541
224	10028
225	17196
226	24605
227	13802
228	17355
229	28629
230	17394
231	20361
232	22816
233	12701
234	9623
235	25637
236	10867
237	6125
238	8124
239	13672
240	51180
241	34488
242	1593
243	880
244	478
245	254
246	152
247	104
248	66
249	45
250	31
251	17
252	24
253	19
254	16
255	1

```
disp('Histogram Values for Quantized Image:');
```

Histogram Values for Quantized Image:

```
disp([quant_bins, hist_quant]);
```

0	179741
1	94168
2	415225
3	66344
4	25193
5	20674
6	19117
7	18575
8	17663
9	16386
10	15208
11	13760
12	13568
13	13110
14	13146
15	13573
16	14161

17	13801
18	13461
19	13427
20	13545
21	13477
22	13774
23	13427
24	12814
25	12797
26	12687
27	12742
28	12791
29	12417
30	12328
31	12293
32	12597
33	12057
34	12272
35	12418
36	12313
37	12125
38	11981
39	12859
40	12749
41	12501
42	12529
43	12358
44	12215
45	12114
46	11778
47	11758
48	11793
49	11692
50	11740
51	11778
52	12172
53	12099
54	11979
55	11803
56	11990
57	12305
58	12485
59	13096
60	13261
61	13519
62	14718
63	15498
64	16955
65	18455
66	20667
67	22004
68	22701
69	22283
70	21901
71	22164
72	21974
73	22185
74	23040
75	24720
76	25451
77	27746
78	27673
79	30136
80	35351

81	35700
82	39016
83	42181
84	46938
85	52516
86	48608
87	50395
88	62118
89	92932
90	78655
91	71760
92	67783
93	66746
94	64744
95	66969
96	69734
97	65264
98	62305
99	60688
100	62273
101	61032
102	61174
103	59884
104	61737
105	61932
106	61739
107	61949
108	62198
109	60955
110	60720
111	59454
112	60323
113	56195
114	54884
115	52223
116	49085
117	49016
118	48145
119	47348
120	45865
121	44901
122	43604
123	42605
124	41772
125	40908
126	37976
127	36432
128	35351
129	34285
130	33044
131	32269
132	33067
133	30145
134	28578
135	27550
136	26611
137	26141
138	25898
139	25897
140	26232
141	26960
142	26885
143	26860
144	26141

145	25932
146	26128
147	24266
148	23063
149	22612
150	23514
151	23718
152	22911
153	20143
154	19454
155	19875
156	20268
157	18856
158	16726
159	15747
160	14161
161	13063
162	12237
163	11212
164	10244
165	10001
166	10051
167	9858
168	10319
169	8811
170	8133
171	7783
172	7342
173	7385
174	7285
175	7363
176	7104
177	6905
178	6454
179	6145
180	5858
181	5830
182	5785
183	5729
184	5871
185	5831
186	5792
187	5974
188	6090
189	5944
190	5767
191	5883
192	5918
193	5873
194	5847
195	6052
196	6154
197	6114
198	6086
199	6159
200	6199
201	6044
202	6335
203	6289
204	5946
205	5849
206	5869
207	5841
208	5600

209	5792
210	5836
211	6135
212	6230
213	6443
214	6718
215	7259
216	7921
217	7556
218	7432
219	8049
220	8562
221	8840
222	9246
223	10061
224	10735
225	10758
226	14176
227	13765
228	21867
229	20421
230	15802
231	13557
232	11772
233	11544
234	12052
235	15677
236	9954
237	4369
238	3206
239	3418
240	10130
241	13406
242	8365
243	5030
244	4290
245	3877
246	2726
247	1801
248	1326
249	815
250	634
251	445
252	178
253	199
254	21
255	0

```
% Explanation
```

```
disp('Explanation of the Quantization Process:');
```

Explanation of the Quantization Process:

```
disp('Quantization is the process of reducing the number of colors or  
grayscale levels in an image to a limited set. Here, we reduce the original  
grayscale image to 32 levels.');
```

Quantization is the process of reducing the number of colors or grayscale levels in an image to a limited

```
disp('This is achieved using the imresize function, a method often used  
for image resizing. By first resizing the image to a much lower resolution
```

```
and then back up, we effectively reduce the distinct pixel values due to interpolation.');
```

This is achieved using the `imresize` function, a method often used for image resizing. By first resizing the image to a lower resolution and then resizing it back to the original size, we achieve quantization of the pixel values.

```
disp('The first imresize reduces the resolution, compressing the pixel values into fewer clusters. Resizing it back up scales it to the original size but with fewer unique values, achieving quantization.');
```

The first `imresize` reduces the resolution, compressing the pixel values into fewer clusters. Resizing it back to the original size scales the values back to the original range, but with fewer unique values, achieving quantization.

```
disp('The histograms of the original and quantized images illustrate the reduction in levels. The original image's histogram shows a smooth distribution of grayscale values, while the quantized image's histogram has fewer peaks, indicating fewer grayscale levels.');
```

The histograms of the original and quantized images illustrate the reduction in levels. The original image's histogram shows a smooth distribution of grayscale values, while the quantized image's histogram has fewer peaks, indicating fewer grayscale levels.

```
disp('This technique is particularly useful in applications where memory bandwidth is a constraint, or for stylistic effects in graphics design.');
```

This technique is particularly useful in applications where memory bandwidth is a constraint, or for stylistic effects in graphics design.